

第九章：运行时存储空间管理(1)

运行时存储空间的结构、管理
过程活动记录
抽象地址映射

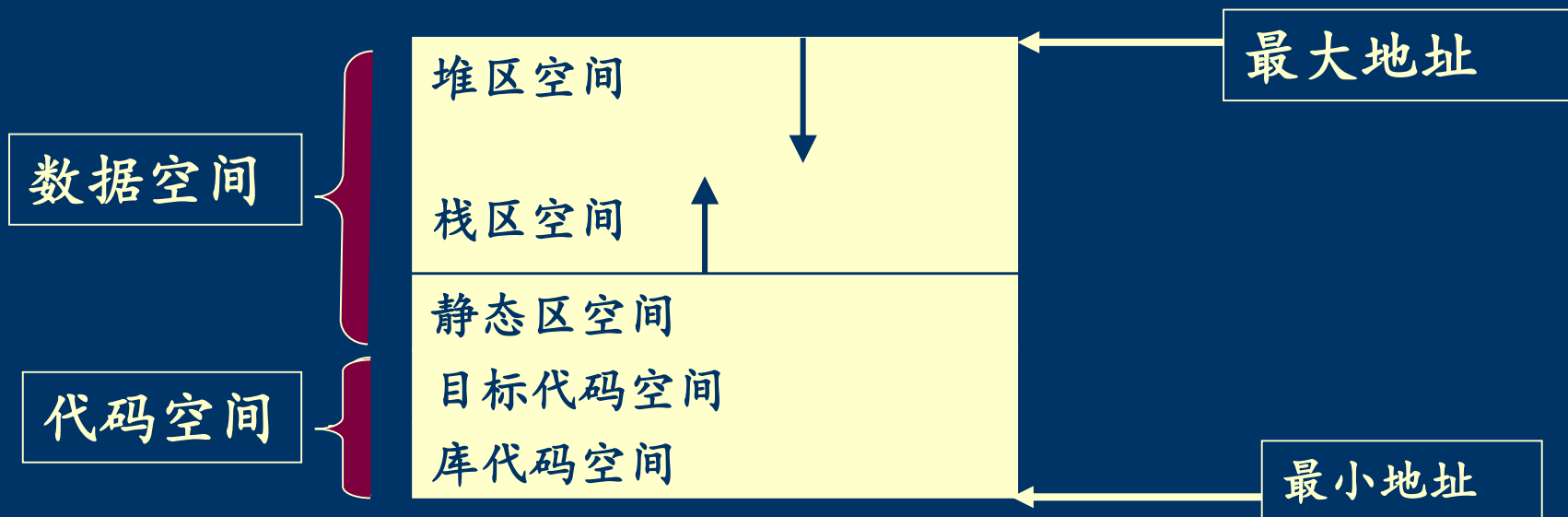
运行时存储空间管理

顾名思义，运行时存储空间的管理，是指目标程序运行时存储空间的管理

- ◆ 本章中要解决两个问题：
- ❖ 运行时空间是怎么划分、分配、回收的
- ❖ 存储空间是如何访问的

目标程序运行时内存的划分

目标程序占用的存储空间如图所示：



目标程序运行时

堆区空间

栈区空间

静态区空间

目标代码空间

库代码空间

- ◆ 库代码区 (library space) : 用于存放标准库函数的目标代码, 例如pascal语言中用到的sin、cos等等, C语言中通过#include包含进来的库代码;
- ◆ 目标代码区 (code space) : 用以存放编译生成的目标程序。实际上一个目标程序变成可执行文件需要一个链接-装配的过程, 链接装配之后存放在目标代码区。

目标程序运行时

堆区空间

栈区空间

静态区空间

目标代码空间

库代码空间

- ◆ 静态区 (static space) : 有些数据对象所占用的空间能在编译时确定, 其地址可以编译进目标代码中, 这些数据对象通常存放在静态区中, 如静态变量和全局变量
- ◆ 栈区 (stack space) : 存放过程活动记录, 该存储区被称作一个栈, 一个元素是一个过程活动记录, 调用函数时压栈, 函数结束时退栈

目标程序运行时

堆区空间

栈区空间

静态区空间

目标代码空间

库代码空间

- ◆ 堆区 (heap space)：堆不是一个连续分配的模式，可以进行动态分配的空间管理，主要用于存放动态申请的数据对象（如C, pascal 等语言的 malloc, free, new, delete）
- ◆ 栈区和堆区之间没有事先划好的界线，当目标代码运行时，栈区指针和堆区指针不断地变化，并朝着对方方向不断增长。如果这两个区相交，则表示出现了内存溢出。

语言影响空间分配策略

- ◆ (1) 语言中是否允许函数、过程的递归调用？
- ◆ 允许递归调用，则函数的形参、局部量可能对应是一系列存储单元，不能采用静态的存储分配方式；
- ◆ 不允许有递归出现，则一个函数最多只能分配一个活动记录大小的空间。

语言影响空间分配策略

- ◆ （2）当函数调用结束时局部变量是否需要保存？
- ◆ 一般来说，一个函数的局部变量在函数调用结束的时候所占用的存储空间都要被释放掉，但是有一类语言的特殊语句，允许函数中的某个局部变量保存着，下一次再调用这个函数的时候，这个局部变量是已经具有上一次语言的值。

语言影响空间分配策略

- ◆ (3) 是否可以访问非局部的变量？比方说外层的变量，还有其他变量是否可以被访问，不同的语言也有不同的约定。

语言影响空间分配策略

- ◆ (4) 函数参数的传递方式
- ◆ 常用的参数传递方式是值引用和地址引用，这就决定了地址引用型实参只能是一个变量，而不能是一个常数，形参的值引用和地址引用的空间分配的方式是不一样的。
 -

语言影响空间分配策略

- ◆ (5) 函数是否可以作为参数进行传递？
- ◆ (6) 函数的结果是否可以作为函数？
- ◆ (7) 动态地申请存储块？
- ◆ (8) 是否显式地释放存储空间？

存储管理模式的分类

静态存储分配策略

动态存储分配策略

栈式动态存储分配策略

堆栈混合式动态存储分配策略

静态分配策略在编译时为数据对象分配固定的存储空间，且存储对象的存储位置在程序的整个生命周期是固定的。

静态存储分配示例

```
int sum = 0;
int square(int i)
{
    int j;
    j = i * i;
    return j;
}
void main()
{
    int j;
    j = square(n);
    sum = sum + j;
}
```

```
(ENTRY, Lsqr, 3+off, 0)
(*, i, i, t1)
(assig, t1, _, j)
(RETURN, _, _, j)
(ENDFUNC, _, _, _)
(ENTRY, Lmain, 3+off, 0)
(VALACT, n, offi, 1)
(call, square, true, t2)
(assig, t2, 1, j)
(+, sum, j, t3)
(assig, t3, 1, sum)
(ENDFUNC, _, _, _)
```

目标代码
sum
常量信息表等...
square 的控制信息
...
i
j
t₁
main 的控制信息
.....
j
t₂
t₃

静态存储分配

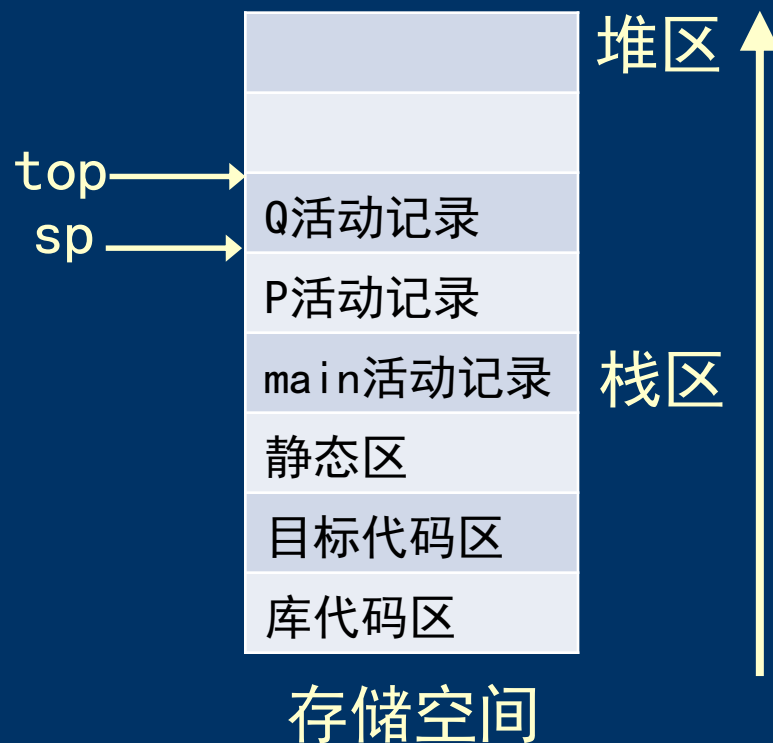
- ◆ 完全采用静态分配策略的语言必须满足以下约束条件：
 1. 不允许有递归过程/函数。
 2. 数据对象的长度和它在内存中位置，必须是在编译时可知。
 3. 不允许动态建立的数据结构（如动态数组、指针等），因为没有运行时的存储分配机制。

动态存储分配

- ◆ 动态存储分配，可以分为：
 - ❖ 栈式。是一种最常见的模式，在有函数调用的时候动态的分配存储空间；
若程序运行过程中需要动态进行申请和释放程序空间，就不能用栈式分配，因为没有办法静态地在活动记录里给动态变量分配空间
 - ❖ 栈堆混合式，活动记录的大小确定的存在栈区中，动态申请的空间在堆区中申请然后释放，同时增加一部分对堆区的管理程序

栈式管理中的过程活动记录

- ◆ 过程活动记录的申请和释放是栈式管理中最重要内容
- ◆ 栈区中通常需要设立两个指针：
 - ❖ sp 指向当前活动记录的起始位置
 - ❖ top 指向第一个可用的存储单元
 - ❖ 通常存在寄存器中

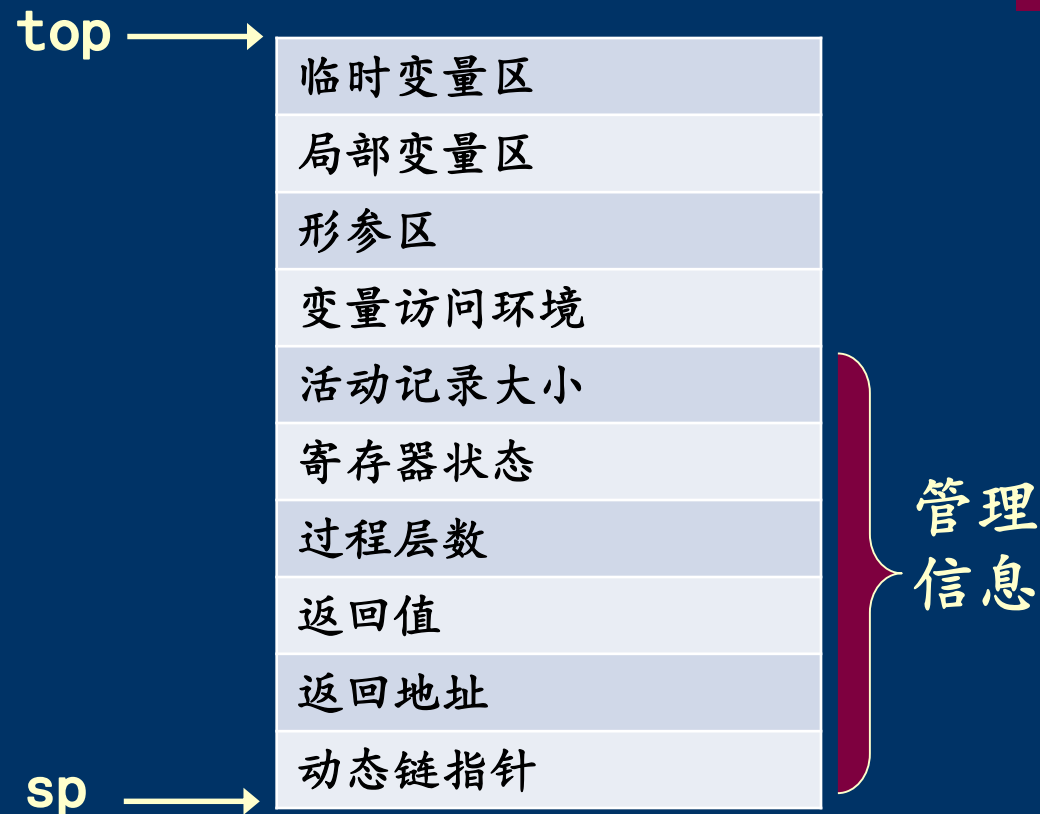


栈式管理中的过程活动记录

- ◆ 过程的活动记录：

为管理过程/函数的一次活动所需要的信息，目标程序要在栈区中给被调过程分配一段连续的存储空间，以便存放该过程的局部变量值、控制信息和寄存器内容等，称这段连续的存储空间为过程的活动记录，简称活动记录（Activation Record），并记为AR。

栈式管理中的过程活动记录



过程活动记录

栈式管理中的过程活动

临时变量区

局部变量区

形参区

变量访问环境

活动记录大小

寄存器状态

过程层数

返回值

返回地址

动态链指针

- (1) **临时变量区** 用于存放表达式计算。寄存器不足以存放所有这些临时变量时，存放在临时变量区中。
- (2) **局部变量区** 保存过程局部声明的数据。
- (3) **形参变量区** 用于存放调用过程提供的实在参数。
- (4) **返回值** 存放一存储空间地址，该存储空间用于存放被调用过程返回给调用过程的值。
- (5) **变量访问环境** 提供非局部变量的访问环境。

栈式管理中的过程活动

(6) **寄存器状态** 保存刚好在过程调用时
信息，包括程序计数器的值和控制从
时必须恢复的机器寄存器的值。

(7) **过程层数** 辅助构建变量访问环境和变量寻址。

(8) **返回地址** 指向调用者调用指令的下一条指令。

临时变量区

局部变量区

形参区

变量访问环境

活动记录大小

寄存器状态

过程层数

返回值

返回地址

动态链指针

栈式管理中的过程活动记录

被调用者的AR空间

(9) **动态链指针** 用来指向调用者的活动记录。该指针也称为控制链。以便在过程调用返回时，活动记录恢复成调用者的活动记录；

(10) **活动记录空间大小** 当过程的局部静态数组等可变长数据时，过程的活动记录空间大小不能静态确定，将其记录下来，可以用于过程调用时确定现行AR的始地址。

注意：活动记录其实并没有包含过程一次执行所需的全部信息，比方说非局部数据就不在活动记录中。另外，过程运行时生成的动态变量也不在活动记录中，对它们通常采用堆式分配。

临时变量区

局部变量区

形参区

变量访问环境

活动记录大小

寄存器状态

过程层数

返回值

返回地址

动态链指针

调用者的AR

过程活动记录的申请和释放

- ◆ 在遇到call四元式时，要生成相应的目标代码，为新的活动记录申请空间。主要工作有：
 - ❖ 产生一个新的活动记录，即 $sp=top$ ， $top=top+size$
 - ❖ 保存寄存器内容，填写过程活动记录的管理信息，返回地址、动态链指针等等
- ◆ 遇到return或遇到endfunc/endproc时，要生成相应的目标代码，收回活动记录空间。主要工作有：
 - ❖ 恢复现场，即恢复各寄存器的值
 - ❖ 释放当前活动记录即 $top=sp$ ； $sp=$ 动态链指针
 - ❖ 根据返回地址生成跳转指令

过程活动记录的申请和释放

调用链：调用链是过程名的序列，序列的头是主程序名M（pascal主程序、C语言的main函数）。具体地说：

- ❖ (M) 是调用链；
 - ❖ 若 (M, ..., R) 是调用链，并且R中有S的调用，则 (M, ..., R, S) 也是调用链。
-
- 对于任一过程（函数）S，其调用链不是唯一的，每个调用链对应于一个动态的过程调用序列。
 - 用CallChain(S) = (M, ..., R, S) 表示S的调用链，表示当前正在执行的是S的过程体，而M, ..., R则是已经开始执行但被中断了的过程。

过程活动记录的申请和释放

动态链：如果当前正在执行的是S，并且

$$CallChain(S) = (M, N, \dots, R, S),$$

则栈的当前内容可表示为：

$$[AR(M), AR(N), \dots, AR(R), AR(S)],$$

称它为对应调用链 $(M, N, \dots, R, S)$ 的动态链。

表示为：

$$\begin{aligned} DynamicChains(S) \\ = [AR(M), AR(N), \dots, AR(R), AR(S)] \end{aligned}$$

调用链和动态链的示例

```

program P;
var a, x : integer;
procedure Q(b: integer);
  var i: integer;
  procedure R(u: integer; v: integer);
  主程序P (P)
  var c, d: integer;
  主程序P 过程 S (P,S)
  begin
    if u=1 then R(u+1, v);
  主程序P 过程 S 过程 Q (P,S,Q)
    v:=(a+c)*(b-d);
  主程序P 过程 S 过程 Q 过程 R (P,S,Q,R)
  end {R};
begin
  主程序P 过程 S 过程 Q 过程 R 过程 R (P,S,Q,R,R)
  R(1,x);
  .....
end {Q}
  
```

调用链

主程序P $\Rightarrow$ 过程 S
 $\Rightarrow$ 过程 Q $\Rightarrow$ 过程 R
 $\Rightarrow$ 过程 R

```

procedure S;
  var c, i:integer;
  begin
    a:=1;
    .....
  end {S}
  主程序P 过程 S 过程 Q 过程 R 过程 R (P,S,Q,R,R)
begin
  a:=0;
  主程序P 过程 S 过程 Q 过程 R 过程 R (P,S,Q,R,R)
  .....
end. {P}
  
```

调用链和动态链的示例

调用链

动态链

主程序P	(P)	[AR(P)]
主程序P	过程 S	(P,S) [AR(P), AR(S)]
主程序P	过程 S	过程 Q (P,S,Q) [AR(P), AR(S), AR(Q)]
主程序P	过程 S	过程 Q 过程 R (P,S,Q,R) [AR(P), AR(S), AR(Q), AR(R)]
主程序P	过程 S	过程 Q 过程 R 过程 R (P,S,Q,R,R) [AR(P), AR(S), AR(Q), AR(R), AR(R)]

主程序P $\Rightarrow$ 过程 S $\Rightarrow$ 过程 Q $\Rightarrow$ 过程 R $\Rightarrow$ 过程 R

空间申请过程



主程序P $\Rightarrow$ 过程 S $\Rightarrow$ 过程 Q $\Rightarrow$ 过程 R $\Rightarrow$ 过程 R

空间释放过程

执行下述指令:

TOP:=SP
SP:=0[SP]



地址分配原则回顾

- ◆ 值引用的形参按照类型大小分
- ◆ 地址引用的形参分配1
- ◆ 局部变量按照类型长度分
- ◆ 临时变量分1（一般情况下）
- ◆ 函数和过程作为形参分2，其中1个是实参的入口地址，另一个是先行的Display表地址

抽象地址的变化规律

1. 处理声明部分:

处理前可用的
抽象地址

处理内容

处理后可用的
抽象地址

(l, off)	→	LabelDec	→	(l, off)
(l, off)	→	ConstDec	→	(l, off)
(l, off)	→	TypeDec	→	(l, off)
(l, off)	→	Var id:T	→	$(l, \text{off} + n_T)$
(l, off)	→	ProcDec	→	(l, off)
(l, off)	→	FuncDec	→	(l, off)

II.处理完函数首部:

处理前可用的 抽象地址	处理内容	处理后可用的 抽象地址
$(l, off) \rightarrow$	Proc p()	$(l+1, off_1)$
$(l, off) \rightarrow$	Func f():T	$(l+1, off_1)$
$(l, off) \rightarrow$	Proc P()	$(l, off+2)$ (形式过程)
$(l, off) \rightarrow$	Func F():T	$(l, off+2)$ (形式函数)

注：小写字母的标识符表示实在标识符，大写字母的标识符则表示形参标识符。 n_T 表示类型T的长度， off_1 表示处理完形参部分（实在过程首部）时的偏移值。

III. 处理形式参数:

$$(\ell, \text{off}) \rightarrow \text{Var ID:T} \rightarrow (\underline{\ell}, \text{off}+1)$$
$$(\ell, \text{off}) \rightarrow \text{ID:T} \rightarrow (\underline{\ell}, \text{off} + n_T)$$

IV. 处理过程名及左括号之后:

$$(\ell, \text{off}) \rightarrow \text{Proc } p(\rightarrow (\ell+1, d)$$
$$(\ell, \text{off}) \rightarrow \text{Func } f(\rightarrow (\ell+1, d)$$
$$(\ell, \text{off}) \rightarrow \text{Proc } P(\rightarrow (\ell+1, d)$$
$$(\ell, \text{off}) \rightarrow \text{Fucn } F(\rightarrow (\ell+1, d)$$

注: 小写字母的标识符表示实在标识符, 大写字母的标识符则表示形参标识符。 n_T 表示类型T的长度, off_1 表示处理完形参部分(实在过程首部)时的偏移值, d表示活动记录控制信息所占的大小。

抽象地址分配的实例：

```
(l, 10) Label 100, 200;  
(l, 10)  
    Const pai=3.14;  
(l, 10)  
    Type arr=array[1..10]of integer;  
(l, 10)  
    Var x:integer;  
(l, 11)  
    a:array[1..5]of integer;  
(l, 16)  
    Function f (  
        (l+1, d)  
            Var x:real;  
        (l+1, d+1)  
            a:arr;  
        (l+1, d+11)  
            Var c:arr;  
        (l+1, d+12)  
            Procedure G ( ) ;  
        (l+1, d+14 )  
            Function F ( ) :real  
        (l+1, d+16 )  
            ) :real;  
        (l+1, d+16 =off1)  
        Begin .....end;  
(l, 16)
```

变量的地址映射

- ◆ 并列式语言：

相对简单只有0层和1层，可以用两个指针来解决，sp0指向全局量的首地址，sp指向当前活动记录的首地址，根据变量的层数来确定是 $sp0+off$ 还是 $sp+off$ 。

例如 `int x, y; int f() {int i; y=x*i; return y;}`

变量的地址映射

- ◆ 嵌套式语言：相比之下复杂一些

若抽象地址是 (L, off) ，如果 L 等于当前层，则说明它所对应的变量处于当前的活动记录中，如果 L 不等于当前层，就要找到它所对应的活动记录的首地址，为此要构造一个 `Display` 表，把每一个活动记录活跃的静态外层的首地址都存起来，然后找到 L 所对应的那一层的首地址加上 `off` 就可以了